Lista de Exercícios 2 - Listas e Dicionários em C#

Prof. Dr. Rodrigo Colnago Contreras
contreras@unifesp.br
Lattes Google Scholar ORCID

Criada em 23/11/2025 e revisada em 09/04/2026 e em 18/09/2026.

Introdução: List e Dictionary em C#

List<T>

A estrutura `List<T>` representa uma lista dinâmica, semelhante a um vetor que cresce e diminui automaticamente. Permite armazenar elementos em ordem e acessá-los por índice.

- Criar: `List<int> numeros = new List<int>();`
- Adicionar: `numeros.Add(10);`
- Acessar: `numeros[0];`
- Remover: `numeros.Remove(10);` ou `numeros.RemoveAt(0);`
- Tamanho: `numeros.Count;`
- Percorrer: `foreach (var n in numeros) { ... }`

Dictionary<TKey, TValue>

A estrutura `Dictionary<TKey, TValue>` funciona como um mapa de pares chave–valor. É útil quando se deseja recuperar informações rapidamente a partir de uma chave.

- Criar: `Dictionary<int, string> alunos = new Dictionary<int, string>();`
- Adicionar: `alunos.Add(1, "João");`
- Acessar: `alunos[1];`
- Modificar: `alunos[1] = "João Silva";`
- Verificar: `alunos.ContainsKey(1);`
- Remover: `alunos.Remove(1);`
- Percorrer: `foreach (var par in alunos) { par.Key; par.Value; }`

Quando usar cada um?

- **List<T>**: quando os elementos são armazenados em ordem e acessados por índice.
- **Dictionary<TKey, TValue>**: quando é necessário associar dados a uma chave e buscar valores de forma rápida.

Exercícios

Listas

Exercício 1. Dada a lista a seguir, encontre o maior e o menor elemento por meio de uma varredura com um laço `for`/`foreach`.

```
// Exemplo de geracao da lista em C#
var rand = new Random();
List<double> lista_exercicio_1 = new List<double>();

for (int i = 0; i < 1000; i++)
    lista_exercicio_1.Add(rand.NextDouble() * 2 - 1); // entre -1 e 1
```

Exercício 2. Dada a lista a seguir, separe-a em duas listas: uma contendo apenas os elementos pares e outra contendo apenas os ímpares.

```
var rand = new Random();
List<int> lista_exercicio_2 = new List<int>();

for (int i = 0; i < 1000; i++)
    lista_exercicio_2.Add(Math.Abs((int)(100 * (rand.NextDouble() * 2 - 1))
       ));
```

Exercício 3. Escreva um programa que leia um valor inteiro positivo n. Em seguida, o programa deve ler n valores numéricos e anexá-los a uma lista. Ao final, apresente:

- a lista preenchida;
- a soma dos valores;
- a média dos valores;
- o menor e o maior valor digitado.

Exercício 4. Dada uma lista `lista_original`, crie uma lista "espelhada", ou seja, com a ordem dos elementos invertida.

```
var rand = new Random();
List<int> lista_original = new List<int>();

for (int i = 0; i < 1000; i++)
    lista_original.Add(Math.Abs((int)(100 * (rand.NextDouble() * 2 - 1))));
```

Exercício 5. Escreva um programa que faça a soma convexa entre duas listas de mesmo tamanho. Isto é, sejam:

$$L_1 = [l_{1,1}, \ldots, l_{1,n}] \qquad L_2 = [l_{2,1}, \ldots, l_{2,n}]$$

O usuário deve fornecer n, duas listas e um valor $\alpha \in [0, 1]$, e a lista resultante deve ser $L_3 = [l_{3,1}, \ldots, l_{3,n}]$ tal que:

$$L_{3,i} = \alpha L_{1,i} + (1 - \alpha) L_{2,i}, i = 1, 2, ..., n.$$

Exercício 6. Uma forma de cruzar informações entre duas listas é conduzir uma mistura de coordenadas. Uma das maneiras mais comuns de se executar este processo é definindo o que se intitula ponto de corte na lista, que é um valor inteiro entre 0 e o tamanho da lista. Na sequência, secciona-se as listas de acordo com este ponto de corte e confecciona-se duas listas resultantes com a união das partes diferentes geradas em cada lista. A seguir, apresenta-se um exemplo com duas listas `lista_1` e `lista_2` de tamanho 6, considerando-se um ponto de corte = 2, geram-se com o processo descrito duas listas resultantes `mesc_1` e `mesc_2`, tal como exemplificado na Figura 1.

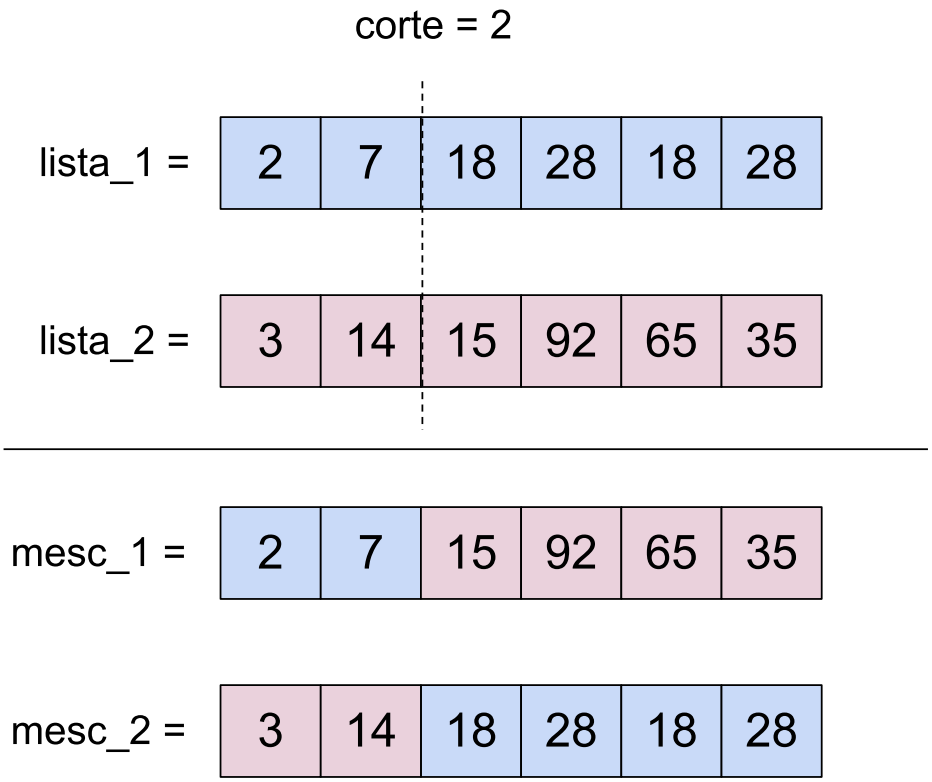


Figura 1: Exemplo do procedimento conhecido na literatura como *Partially Mapped Crossover*.

Escreva um programa que leia um ponto de corte do usuário e que, dadas duas listas `lista_1` e `lista_2` abaixo, gere através do processo de cruzamento de informação mencionado duas listas resultantes `mesc_1` e `mesc_2`.

```
var rand = new Random();
List<int> lista_1 = new List<int>();
List<int> lista_2 = new List<int>();

for (int i = 0; i < 1000; i++) {
    lista_1.Add(rand.Next(0, 100));
    lista_2.Add(rand.Next(0, 100));
}
```

Exercício 7. Escreva um programa em C# que, dada uma lista como a definida na sequência de código a seguir e dados dois índices i e j dessas listas, conduza as seguintes perturbações nessa lista:

swap O elemento da posição i deve ser alocado na posição j e o elemento da posição j deve ser alocado na posição i. Isto é, ambos os elementos devem trocar de posição entre si.

inverse A sublista definida pelos elementos internos às posições i e j deve ser invertida.

insert Supondo que $j > i$, o elemento da posição j deve ser alocado na posição i, o elemento da posição i deve ser alocado na posição $i + 1$ e os elementos da sublista definida entre às antigas posições i e j devem ser transladados em uma posição.

Obs.: Em otimização computacional, é muito comum o uso de simulações de mutações cromossômicas. Especificamente, dada uma lista c de n elementos, uma mutação consiste em um procedimento que perturba c tendo como base dois índices de c. Em outras palavras, uma função f é uma mutação em c se, e somente se, $f(c, (i, j))$ é uma possível permutação dos elementos de c. Na Figura 2, são apresentadas 3 possíveis mutações: *swap*, *inverse* e *insert*.

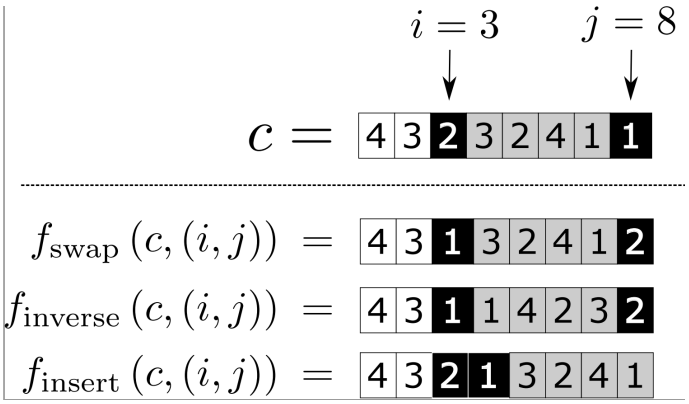


Figura 2: Exemplo do funcionamento de mutações.

```
var rand = new Random();
List<int> c = new List<int>();

for (int i = 0; i < 10; i++)
    c.Add(Math.Abs((int)(100 * (rand.NextDouble() * 2 - 1))));

int i_index = rand.Next(0, c.Count);
int j_index = rand.Next(0, c.Count);
```

Exercício 8. Escreva um programa em C# que leia dois valores inteiros positivos: `numero_de_alunos` e `numero_de_provas_por_aluno`. Essas duas variáveis descrevem, respectivamente, o número de alunos em uma turma de matemática e o número de provas aplicadas pelo professor nessa turma.

Seu programa precisa construir uma lista para cada um desses alunos contendo o nome do aluno (digitado pelo professor); os valores de cada uma das `numero_de_provas_por_aluno` provas e a média entre essas provas para cada aluno. Assim, cada uma dessas listas terá tamanho `numero_de_provas_por_aluno + 2`. Além disso, cada uma dessas listas deverá ser anexada em uma outra lista, chamada de `lista_de_notas_dos_alunos`.

Ao final do programa, apresente a lista `lista_de_notas_dos_alunos`.

Exercício 9. Considere a variável `lista_mestra` composta por várias listas numéricas, definida na sequência. Escreva um programa que encontre:

- o maior elemento entre **todas** as listas;
- o menor elemento entre **todas** as listas.

```
var rand = new Random();
List<List<double>> lista_mestra = new List<List<double>>();

int numero_de_listas = 100;
int tamanho = 10;

for (int k = 0; k < numero_de_listas; k++) {
    var lista = new List<double>();
    for (int i = 0; i < tamanho; i++)
        lista.Add(Math.Round((rand.NextDouble() * 2 - 1) * 100) / 100);
    lista_mestra.Add(lista);
}
```

Exercício 10. Usando a mesma variável `lista_mestra`, escreva um programa que conte, para cada lista, quantos elementos são positivos e quantos são negativos. Armazene os resultados em uma nova lista contendo tuplas:

$$(\text{positivos}_k, \text{negativos}_k)$$

para cada lista k.

Dicionários

Exercício 11. Para cada linha da tabela a seguir, construa um dicionário em C# (usando `Dictionary<string, object>`) cujas chaves sejam "Nome", "Grupo", "Período", "Número Atômico" e "Massa Atômica", com os valores correspondentes. Na sequência, cadastre os elementos correspondentes:

Nome	Grupo	Período	Número Atômico	Massa Atômica
Ouro	11	6	79	197
Prata	11	5	47	108
Hélio	18	1	2	4
Cloro	17	3	17	35

Exercício 12. Modifique o valor da chave "Nome" dos dicionários criados no exercício anterior para os símbolos químicos: Ouro → Au, Prata → Ag, Hélio → He, Cloro → Cl.

Exercício 13. Crie uma nova chave "Estado físico (CNTP)" nos dicionários e preencha com: Au → "Sólido", Ag → "Sólido", He → "Gasoso", Cl → "Gasoso".

Exercício 14. Remova a chave "Grupo" dos dicionários criados.

Exercício 15. Escreva um programa que crie uma `List<Dictionary<string,object>>` com `n` dicionários (mesmas chaves do exercício 1), onde `n` é um valor inteiro positivo digitado pelo usuário. O preenchimento deve ser feito via `Console.ReadLine`. Ao final, apresente todos os dicionários digitados.

Importante: Considere a lista de dicionários a seguir para responder aos próximos 5 exercícios:

```
1
2       List<Dictionary<string, object>> lista_de_dicionarios =
3           new List<Dictionary<string, object>>
4       {
5           new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",72}},
6           new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Mercedes"}, {"idade",22}},
7           new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
               segundo_nome","Bracho"}, {"idade",60}},
8           new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Mercedes"}, {"idade",91}},
9           new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Bracho"}, {"idade",35}},
10          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",26}},
11          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",71}},
12          new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
               segundo_nome","Bracho"}, {"idade",73}},
13          new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
               segundo_nome","Bracho"}, {"idade",72}},
14          new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Bracho"}, {"idade",83}},
15          new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Mercedes"}, {"idade",63}},
16          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Bracho"}, {"idade",21}},
17          new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Bracho"}, {"idade",86}},
18          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",69}},
19          new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
               segundo_nome","Mercedes"}, {"idade",80}},
20          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Bracho"}, {"idade",87}},
21          new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",61}},
22          new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
               segundo_nome","Perez Ochoa"}, {"idade",21}},
23          new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
               segundo_nome","Mercedes"}, {"idade",56}},
```

```
24            new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
                  segundo_nome","Bracho"}, {"idade",24}},
25            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Mercedes"}, {"idade",70}},
26            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Perez Ochoa"}, {"idade",69}},
27            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Bracho"}, {"idade",94}},
28            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Bracho"}, {"idade",35}},
29            new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
                  segundo_nome","Bracho"}, {"idade",54}},
30            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Mercedes"}, {"idade",66}},
31            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Perez Ochoa"}, {"idade",83}},
32            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Perez Ochoa"}, {"idade",26}},
33            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Mercedes"}, {"idade",49}},
34            new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
                  segundo_nome","Bracho"}, {"idade",57}},
35            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Bracho"}, {"idade",97}},
36            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Bracho"}, {"idade",42}},
37            new Dictionary<string, object>{{"primeiro_nome","Paola"}, {"
                  segundo_nome","Mercedes"}, {"idade",82}},
38            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Bracho"}, {"idade",56}},
39            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Mercedes"}, {"idade",29}},
40            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Mercedes"}, {"idade",29}},
41            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Bracho"}, {"idade",19}},
42            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Mercedes"}, {"idade",90}},
43            new Dictionary<string, object>{{"primeiro_nome","Rubi"}, {"
                  segundo_nome","Perez Ochoa"}, {"idade",52}},
44            new Dictionary<string, object>{{"primeiro_nome","Maria"}, {"
                  segundo_nome","Bracho"}, {"idade",71}}
45        };
46
47        Console.WriteLine("Lista carregada com " + lista_de_dicionarios.
              Count + " dicionarios.");
```

Exercício 16. Escreva código em C# que calcule a média das idades, a idade máxima e a idade mínima entre os itens dos dicionários criados acima.

Exercício 17. Escreva código que encontre a concatenação `primeiro_nome + segundo_nome`

com maior número de caracteres entre todos os dicionários e apresente a concatenação, seu comprimento e seu tipo.

Exercício 18. Acrescente em cada dicionário uma chave "categoria" cujo valor seja: "Mocinha" se `primeiro_nome == "Maria"`; caso contrário, "Vilã".

Exercício 19. Verifique quantas repetições existem da configuração `primeiro_nome == "Paola"` e `segundo_nome == "Bracho"` na lista.

Exercício 20. Crie uma nova lista de dicionários a partir da original, sem duplicatas quanto à combinação simultânea de `primeiro_nome` e `segundo_nome` — ou seja, mantenha apenas um representante por par (primeiro, segundo).